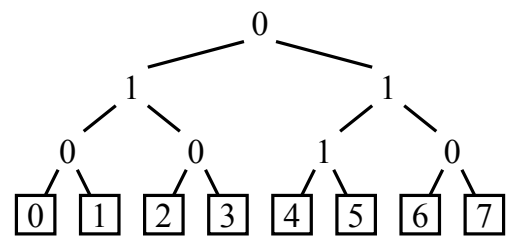


An n -way true least-recently-used (LRU) cache would require $\lceil \log_2(n!) \rceil$ bits of state and more logic than we wish to handle for a fast cache. Many caches instead use a pseudo-LRU (PLRU) system with state that requires fewer gates to access and update. This page describes one such PLRU: the Tree-PLRU.

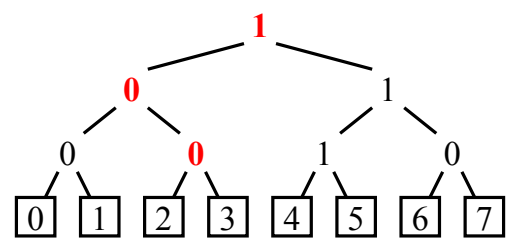
1. Full Binary Tree of Bits

The state stored by a PLRU is a full binary tree of bits, with lines as the leaf nodes. The bits indicate which subtree has the PLRU leaf: 0 for the left-hand tree, 1 for the right-hand tree. Thus, in this tree:



Line number 2 of the set is the PLRU set: left (0) at the root, then right (1), then left (0).

When we access a line, we need to change the PLRU data. We do this by setting every bit on the path to the access to point away from that access. For example, if we accessed line number 3 in the previous example we'd now have:



with the new PLRU being line number 6.

Exercise —
Suppose the bits of an 8-way Tree-PLRU are all 0s, meaning the PLRU line is number 0, and we access line number 0. What is the shortest possible access sequence that will again make line number 0 the PLRU line?
The answer is in a footnote.¹

Aside —
Each leaf in this tree can be categorized by the number of bits you'd need to flip in order to make it the PLRU element. In our first example tree, that is

In a sense, the “1 violation” leaves are all lumped into one equivalence class of “next-to-least recently used” and any of them can become the next PLRU with a single access.

2. State bits

For a w -way set, where w is a power of 2, we need $w - 1$ bits of state (the interior nodes in the tree). With those bits, we need to efficiently perform two operations:

- Given the state bits, identify which line to replace (i.e., the PLRU line).
- Given a line access, update the state bits to make that line the most recently used.

2.1. Encoding the bits

We'll store the bits of state as a binary number with the root bit as the low-order bit, it's left child next, and so on.

Example —

Ignoring the leaf nodes, $0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 = 0b0100110 = 0x26$

There are obviously other orders in which we could store these bits, but this order will make handling larger ways simpler.

2.2. Identifying the PLRU

The PLRU line number depends on just one path through the state bits.

We can build the line number for any way by starting with 0 (correct for a 1-way set, i.e. direct-mapped) and traversing down the tree: at each level we double the line number and add the bit we find.

Example —

When navigating $0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7$ we start at 0; $0 \times 2 + 1 = 1$; $1 \times 2 + 1 = 3$; $3 \times 2 + 0 = 6$ which is the PLRU line number.

Navigating the tree is similar, with an extra +1: we start at bit 0 (i.e. the 2^0 's place, the low-order bit) and the bit index after i when reading bit b is $2i + 1 + b$.

Example —

Consider $0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 = 0b0100011$.
We start at bit 0, where we find a 1 (0b0100111); then bit $2 \times 0 + 1 + 1 = 2$, where we find a 0 (0b0100011); then bit $2 \times 2 + 1 + 0 = 5$, where we find a 1 (0b0100011).

2.3. Making a line the most-recently used

The bits of a line number tell us how we'd have to navigate the tree to find that line: the high-order bit is the required root tree bit, etc. To mark the line as most-recently used we navigate the tree according to those bits setting each tree node's bit to the opposite of the corresponding bit of the line number.

Example —

Suppose we want to make line 5 the least-recently used in the following state: $5 = 0b101$ and we work high- to low-order bit

$0b101$, so we set the root to 0 and go right

$0b101$, so we set the node to 1 and go left

$0b101$, so we set the node to 0 and go right

1. After accessing line 0, three bits are set to 1 that all need setting back to 0 before line 0 will again be the PLRU line. We can flip the last of these by accessing line 1, the next by accessing line 2 or line 3, and the last by accessing any of lines 4 through 7.
Thus, one example access sequence that will work is 0 (given), 1, 2, 4.

